

SHL Assessment Recommender

Approach Document

Candidate: Sindhura Karumuri | **Role:** AI Intern | **Repo:** github.com/Sindhura-Karumuri/Recommendation-ChatBot

1. Problem & Design Goals

The task is to build a conversational agent that guides hiring managers from a vague role description to a grounded shortlist of SHL Individual Test Solution assessments (377 items). Core challenges: (1) retrieval quality over a small but domain-specific catalog, (2) conversational control — knowing when to clarify, recommend, refine, or compare, and (3) strict schema compliance for automated evaluation.

2. System Architecture

User → React UI → FastAPI /chat → Agent → Retrieval → LLM → Validated Response
■ Refusal guard (pre-LLM regex)

Stack:

Layer	Choice	Reason
API	FastAPI + Pydantic	Strict schema enforcement, async
LLM	Groq llama-3.3-70b-versatile	Free tier, ~200 tok/s, <1s latency
Fallback LLMs	llama-3.1-8b-instant → gemma2-9b-it	Separate rate-limit quotas
Retrieval	BM25 + TF-IDF + RRF	No cold-start cost, no external deps
Frontend	React + Vite	Served as static files from FastAPI
Deployment	Render (Docker, single service)	Free tier, auto-deploy

3. Retrieval Setup

Hybrid BM25 + TF-IDF with Reciprocal Rank Fusion (RRF). Each catalog item is indexed as a rich text document: name + description + test type labels + job levels + languages + duration. Both BM25 (exact token match) and TF-IDF bigrams (phrase matching) run in parallel over 60 candidates each; RRF merges the rank lists without score normalization tuning.

Why not dense embeddings? At 377 items, dense retrieval adds ~400 MB model weight and significant cold-start latency. BM25+TF-IDF indexes in ~0.3s, handles exact names well ("Java 8", "OPQ32r"), and TF-IDF bigrams cover multi-word concepts ("data science", "machine learning"). Retrieval quality on the catalog was sufficient without embeddings.

Query construction: The retrieval query is built from the last 8 conversation messages concatenated, so refinements ("also add personality tests") naturally shift the retrieved context window.

4. Agent & Prompt Design

****System prompt rules**** (enforced in every turn):

- Clarify before recommending — vague queries require at least one question
- Recommend 1–10 items once role + one constraint is established
- Refine the shortlist on constraint changes, never restart
- Compare using catalog data only
- Refuse off-topic: salary, legal, competitors, prompt injection

****Catalog context injection:**** Top-20 retrieved assessments are formatted as a structured block (name, URL, type codes, job levels, duration, description excerpt) and injected as a second system message every turn. The LLM is instructed to use only these items.

****Turn awareness:**** At turn 5+, a system note pushes the agent to commit to recommendations rather than ask more questions — ensures shortlist delivery before the 8-turn cap.

****JSON-only output:**** ``response_format: json_object`` eliminates markdown-fence parsing fragility.

****Pre-LLM refusal guard:**** Regex patterns catch prompt injection, salary questions, legal questions, and competitor mentions before any API call — guaranteeing zero hallucination on those paths.

****Post-LLM validation:**** Every recommendation is cross-checked against the catalog by name lookup. Non-SHL URLs are patched or dropped entirely. LLM can never return a hallucinated URL.

5. Evaluation Methods

****Schema compliance (hard eval):**** Every response is validated by Pydantic against the exact spec schema. ``tests/test_chat.py`` asserts ``reply: str`, `recommendations: list`, `end_of_conversation: bool`` on every call.

****Behavior probes**** (binary pass/fail, run via ``tests/test_chat.py``):

- No recommendations on vague turn-1 query
- Off-topic refusal (prompt injection, salary)
- All returned URLs are valid SHL catalog URLs (``shl.com/products/product-catalog``)
- Recommendations capped at ≤ 10
- Refinement updates the existing shortlist

****Recall@10 (manual):**** Verified retrieval for 5 representative roles (Java developer, Python data scientist, senior sales manager, front-line supervisor, graduate entry-level). Top-10 retrieved items were checked against expected relevant assessments. BM25+TF-IDF consistently ranked known-relevant items in positions 1–5 for specific queries and 3–8 for broader role queries.

****Multi-turn trace:**** A 5-turn simulated conversation (vague → clarify → refine → compare → close) validated that EOC fires correctly, schema holds throughout, and all URLs come from the catalog.

6. What Didn't Work & How It Was Fixed

Problem	Fix
Static scraping of SHL catalog (JS-rendered)	Used the provided catalog JSON endpoint directly
sentence-transformers + FAISS → 400 MB cold start, exceeded 2-min timeout	Replaced with BM25+TF-IDF, loads in 0.3s

Temperature 0.3 → hallucinated assessment names	Dropped to 0.1, outputs became catalog-grounded
Primary LLM (llama-3.3-70b) daily token limit hit	Added fallback chain: llama-3.1-8b-instant → gemma2-9b-it

7. AI Tools Used

Kiro (agentic coding assistant by AWS) was used for scaffolding, file generation, iterative refactoring, and test writing. All design decisions (retrieval strategy, prompt structure, evaluation approach) were made and validated by the candidate. The code reflects actual understanding and was verified through live testing and behavior probes.

8. Deployment Notes

- Single Render service (Docker multi-stage build: Node → Python)
- FastAPI serves both the API (`/health``, `/chat``) and the React UI (static files at `/``)
- Cold-start delay: ~60–90s on Render free tier (within the 2-minute allowance)
- Primary LLM: **llama-3.3-70b-versatile** via Groq API